

Relatório Final - JavaLibrary

Sistema de Gerenciamento de Biblioteca

Disciplina:	Programação Orientada a Objetos
Professor:	Dilvan de Abreu Moreira
Turma:	SCC0504
Projeto:	JavaLibrary
Integrante:	Demily Estefany Rodrigues da Silva - NUSP: 14710759

1. Requisitos

O projeto desenvolvido foi um sistema desktop simples para gerenciamento de biblioteca. A aplicação atende aos requisitos principais do projeto compacto da turma SCC0504: cadastro de livros, cadastro de usuários, controle de empréstimos, devoluções, busca de registros, persistência em arquivos e tratamento de erros.

- Livro com título, autor, ISBN e quantidade de cópias disponíveis.
- Usuário com ID, nome e contato.
- Empréstimo ligando um livro a um usuário, com data de empréstimo e prazo padrão de 14 dias.
- Interface gráfica com abas separadas para livros, usuários e empréstimos.
- Salvamento e carregamento dos dados usando arquivos de texto.
- Exceções próprias para regras como livro indisponível e empréstimo ativo.

2. Descrição do projeto

O JavaLibrary permite cadastrar livros e usuários da biblioteca, além de controlar empréstimos e devoluções. O sistema foi construído em Java usando Swing para a interface gráfica. Os dados são salvos em arquivos de texto dentro da pasta data, permitindo que as informações continuem disponíveis depois que o programa é fechado.

A aplicação foi organizada em pacotes para separar melhor as responsabilidades: model para as classes principais, service para as regras da biblioteca, persistence para leitura e escrita de arquivos, exception para exceções próprias e view para as telas da interface.

3. Comentários sobre o código

As classes Book, Patron e Loan representam os principais objetos do sistema. A classe Library centraliza as regras de negócio, como impedir ISBN duplicado, impedir ID duplicado, bloquear remoção de registros com empréstimos ativos e atualizar a quantidade de cópias durante empréstimos e devoluções.

O projeto utiliza encapsulamento com atributos privados, construtores, getters e setters, listas de objetos, associação entre classes e herança nas exceções. A interface gráfica chama os métodos da classe Library, evitando colocar regras importantes diretamente nas telas.

4. Plano de testes

Os testes foram feitos manualmente pela interface gráfica, seguindo os principais fluxos do sistema.

1. Cadastrar livro e verificar se aparece na tabela.
2. Cadastrar usuário e verificar se aparece na tabela.
3. Fazer empréstimo e verificar se a quantidade de cópias diminui.
4. Tentar emprestar livro sem cópias disponíveis.
5. Devolver livro e verificar se a quantidade de cópias aumenta.
6. Tentar remover livro ou usuário com empréstimo ativo.
7. Testar ISBN duplicado, ID duplicado, campos vazios e cópias inválidas.

5. Resultados dos testes

Todos os testes manuais executados passaram. O sistema cadastrou livros e usuários corretamente, realizou empréstimos e devoluções, atualizou a quantidade de cópias e exibiu mensagens de erro nos casos inválidos. Os testes estão registrados com mais detalhes no arquivo docs/testes-manuais.md.

6. Procedimentos de build e execução

O projeto não usa bibliotecas externas. Para compilar e executar no Windows, dentro da pasta do projeto, podem ser usados os comandos:

```
javac -d out (Get-ChildItem -Recurse src -Filter *.java).FullName
```

```
java -cp out javalibrary.Main
```

No Linux, os comandos equivalentes são:

```
mkdir -p out
```

```
javac -d out $(find src -name "*.java")
```

```
java -cp out javalibrary.Main
```

7. Problemas encontrados

Não foram encontrados problemas que impedissem a conclusão do projeto. Durante o desenvolvimento, foram feitos apenas ajustes normais de implementação, principalmente na organização da interface gráfica, na validação das operações do sistema e na organização geral do projeto. Também procurei uma boa forma de separar as responsabilidades das classes, deixando modelo, serviço, persistência e interface em partes diferentes, para que o código ficasse mais claro e mais fácil de entender.

8. Comentários finais

O projeto permitiu aplicar conceitos de Programação Orientada a Objetos em uma aplicação completa, com modelo de dados, regras de negócio, persistência em arquivos, tratamento de exceções e interface gráfica. O desenvolvimento foi feito em etapas, com commits separados para mostrar a evolução do sistema.